Deep and Un-Deep Visual Inertial Odometry

Aryan Shah
Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA
Email: ajshah1@wpi.edu

Anirudh Nallawar
Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA
Email: arnallawar@wpi.edu

***Abstract*—Phase 1 focuses on the implementation of a filter-based stereo Vision-aided Odometry using Multi-state Constraint Kalman Filter (MSCKF). This project involves the approach that utilizes sensor fusion from a stereo camera and an IMU. We have implemented seven functions from MSCKF. With the data from these two sensors, we aim to accurately determine the state and localization of the robot. We also have evaluated output of S-MSCKF with respect to ground truth for EuRoC dataset. Index: Multi-state Constraint Kalman Filter, Stereo Camera, IMU**

I. Classical Approach

A. Initialize Gravity and Bias

The 6-DOF IMU sensor used to measure rotation (gyroscope) and acceleration (accelerometer) measurements requires calibration to remove biases. This involves taking the mean of stationary readings to determine the bias, which is then subtracted from subsequent readings. The gyroscope should ideally read [0, 0, 0] but may have small fluctuations due to unknown noise. Similarly, the accelerometer should ideally read [0, 0, -g] in the world frame, but may also have fluctuations due to noise and bias. Calibration is performed before the start of flight to remove biases in both gyroscope and accelerometer readings. The function "initialize gravity and bias" initializes the gravity and bias of the IMU, as well as the initial orientation of the robot, based on the first few IMU readings. Gyro bias and gravity estimate of the IMU are obtained by averaging angular velocity and linear acceleration values coming from the IMU data, respectively. The initial orientation is chosen such that it coincides with the inertial frame of reference. This way, a solid foundation is provided for the ViO process.

- Gravity is normalized to the measured magnitude (`IMUState.gravity` $= [0, 0, -\|\mathbf{g}_{\text{imu}}\|]$), not hard-coded to 9.81, the static-period accelerometer reading sets the magnitude (`msckf.py:250--251`).
- Initial orientation uses `from_two_vectors`$(-\mathbf{g}, \mathbf{g}_{\text{imu}})$, which under the JPL convention gives a quaternion mapping world $\rightarrow$ IMU (not "consistent with the inertial frame," which is vague).

B. Batch IMU Processing

The IMU batch processing function is used to read IMU messages until the next set of images is available from the stereo camera. The state vector used for estimating the next states consists of states related to camera and IMU, including quaternion for rotation, biases for gyroscope and accelerometer, positions, and velocities.

$$\mathbf{x}_I = \begin{pmatrix} {}^I_G\mathbf{q}^\top & \mathbf{b}_g^\top & {}^G\mathbf{v}_I^\top & \mathbf{b}_a^\top & {}^G\mathbf{p}_I^\top & {}^I\mathbf{q}_C^\top & {}^I\mathbf{p}_C^\top \end{pmatrix}^\top$$

The function "batch imu processing"plays an important role in the implemented Visual-Inertial Odometry system. It is responsible for the synchronization of the IMU and VO components.It propagates the state of the IMU by processing the IMU measurements into a buffer within a specified time bound while discarding the already processed messages. For each unprocessed IMU message, the function applies the process model to update the IMU state based on angular velocity and linear acceleration measurements. The IMU state's timestamp and ID are updated, and the processed IMU messages are removed from the buffer.

C. Process Model

The `process_model` function evolves the IMU state along with its covariances based on one IMU sample with bias compensation applied to it. The process model extracts the IMU state including its orientation, velocity, position, and biases of the gyroscope/accelerometer sensor, as well as determines the time step using the IMU measurement timestamp.

It constructs the 21×21 continuous-time error-state transition matrix $\mathbf{F}$ and the 21×12 noise input matrix $\mathbf{G}$, then forms the discrete transition matrix $\mathbf{\Phi}$ as a third-order Taylor expansion of the matrix exponential of $\mathbf{F}\,dt$:

$$\mathbf{\Phi} \approx \mathbf{I} + (\mathbf{F}\,dt) + \tfrac{1}{2}(\mathbf{F}\,dt)^2 + \tfrac{1}{6}(\mathbf{F}\,dt)^3$$

Once $\mathbf{\Phi}$ is built, the state itself is propagated by `predict_new_state` (4th-order Runge–Kutta for velocity and position; closed-form quaternion exponential for orientation). After the new state is in place, three blocks of $\mathbf{\Phi}$ — the rows corresponding to $\delta\boldsymbol{\theta}$, $\delta\mathbf{v}$, and $\delta\mathbf{p}$ against the $\delta\boldsymbol{\theta}$ columns — are rewritten using the previous linearization point (`orientation_null`, `velocity_null`, `position_null`) following the OC-EKF recipe of Hesch et al. This keeps the four unobservable directions (global position and yaw about gravity) in the null space of the observability matrix and prevents the filter from gaining spurious information about them.

The discrete process-noise covariance $\mathbf{Q}$ is then assembled as

$$\mathbf{Q} = \mathbf{\Phi}\,\mathbf{G}\,\mathbf{N}_c\,\mathbf{G}^\top\,\mathbf{\Phi}^\top\,dt,$$

where $\mathbf{N}_c$ is the continuous-time noise covariance. The state covariance $\mathbf{P}$ (not $\mathbf{Q}$ — $\mathbf{Q}$ is the process noise) is updated as

$$\mathbf{P}_{1:21,\,1:21} \leftarrow \mathbf{\Phi}\,\mathbf{P}_{1:21,\,1:21}\,\mathbf{\Phi}^\top + \mathbf{Q}.$$

When camera states are present, the off-diagonal IMU/camera covariance blocks are updated as

$$\mathbf{P}_{1:21,\,22:} \leftarrow \mathbf{\Phi}\,\mathbf{P}_{1:21,\,22:}, \qquad \mathbf{P}_{22:,\,1:21} \leftarrow \mathbf{P}_{22:,\,1:21}\,\mathbf{\Phi}^\top.$$

Finally, $\mathbf{P}$ is symmetrized as

$$\mathbf{P} \leftarrow \tfrac{1}{2}(\mathbf{P} + \mathbf{P}^\top)$$

to suppress numerical drift, and the null-space references (`orientation_null`, `velocity_null`, `position_null`) are reset to the freshly propagated state for use on the next step.

$$\delta\mathbf{x} = \begin{bmatrix}\delta\boldsymbol{\theta} & \delta\mathbf{b}_g & \delta\mathbf{v} & \delta\mathbf{b}_a & \delta\mathbf{p} & \delta\boldsymbol{\theta}_{\text{ext}} & \delta\mathbf{p}_{\text{ext}}\end{bmatrix}^\top$$

$$\mathbf{n} = \begin{bmatrix}\mathbf{n}_g & \mathbf{n}_{wg} & \mathbf{n}_a & \mathbf{n}_{wa}\end{bmatrix}^\top$$

$$\mathbf{C} = \mathbf{C}(^q_G), \quad \hat{\boldsymbol{\omega}} = \boldsymbol{\omega}_m - \mathbf{b}_g, \quad \hat{\mathbf{a}} = \mathbf{a}_m - \mathbf{b}_a$$

$$\mathbf{F} = \begin{bmatrix} -\lfloor\hat{\boldsymbol{\omega}}\times\rfloor & -\mathbf{I}_3 & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ -\mathbf{C}^\top\lfloor\hat{\mathbf{a}}\times\rfloor & \mathbf{0} & \mathbf{0} & -\mathbf{C}^\top & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_3 & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}$$

$$\mathbf{G} = \begin{bmatrix} -\mathbf{I}_3 & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & -\mathbf{C}^\top & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I}_3 \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}$$

D. Predict New State

The function `predict_new_state` advances the IMU pose over a single sample interval dt. Orientation and the linear state are integrated with different schemes, each chosen for what works best for the term in question.

The quaternion is propagated with the closed-form solution to

$$\dot{\mathbf{q}} = \tfrac{1}{2}\,\mathbf{\Omega}(\boldsymbol{\omega})\,\mathbf{q}.$$

With $\boldsymbol{\omega}$ treated as constant over dt, this gives

$$\mathbf{q}(t+dt) = \left[\cos\!\left(\tfrac{1}{2}\|\boldsymbol{\omega}\|dt\right)\mathbf{I}_4 + \frac{\sin\!\left(\tfrac{1}{2}\|\boldsymbol{\omega}\|dt\right)}{\|\boldsymbol{\omega}\|}\,\mathbf{\Omega}(\boldsymbol{\omega})\right]\mathbf{q}(t),$$

and an analogous expression at half-step $dt/2$ for use inside the RK4 loop below. A small-$\|\boldsymbol{\omega}\|$ branch falls back to the first-order form

$$\mathbf{q}(t+dt) \approx \left(\mathbf{I}_4 + \tfrac{1}{2}dt\,\mathbf{\Omega}(\boldsymbol{\omega})\right)\mathbf{q}(t)$$

to avoid division by near-zero norms.

Velocity and position are then propagated by fixed-step 4th-order Runge–Kutta (not adaptive) using the accelerometer measurement rotated into the world frame plus gravity:

$$\dot{\mathbf{v}} = \mathbf{R}(\mathbf{q})^\top\mathbf{a} + \mathbf{g}, \qquad \dot{\mathbf{p}} = \mathbf{v}.$$

The four RK4 stages $k_1 \ldots k_4$ evaluate $\dot{\mathbf{v}}$ at the start of the interval (with the current rotation), at the midpoint twice (using the half-step quaternion $\mathbf{R}(\mathbf{q}_{dt/2})^\top$), and at the endpoint (using the full-step quaternion $\mathbf{R}(\mathbf{q}_{dt})^\top$); the position stages take the corresponding intermediate velocities. The state is then updated as

$$\mathbf{v} \leftarrow \mathbf{v} + \frac{dt}{6}\left(k_{1v} + 2k_{2v} + 2k_{3v} + k_{4v}\right),$$

$$\mathbf{p} \leftarrow \mathbf{p} + \frac{dt}{6}\left(k_{1p} + 2k_{2p} + 2k_{3p} + k_{4p}\right),$$

and the orientation is replaced with the normalized full-step quaternion. Splitting the integration this way — closed-form for the quaternion, RK4 for $\mathbf{v}$ and $\mathbf{p}$ — is what the authors' C++ does and matches Eq. (1) of the original MSCKF paper.

E. State Augmentation

The function `state_augmentation` appends a new camera state derived from the current IMU pose every time a new image arrives, and grows the state covariance by the corresponding 6 rows and columns. Given the IMU↔cam0 extrinsics $\mathbf{R}_i^c$ (rotation from IMU to cam0) and ${}^I\mathbf{p}_C$ (origin of cam0 expressed in the IMU frame), the camera pose is computed as

$$\mathbf{R}_w^c = \mathbf{R}_i^c\,\mathbf{R}_w^i, \qquad \mathbf{t}_c^w = {}^G\mathbf{p}_I + \mathbf{R}_w^{i\top}\,{}^I\mathbf{p}_C,$$

then stored in the new `CAMState` along with copies for the OC-EKF null-space reference.

The state-augmentation Jacobian is the (6×21) block $\mathbf{J}$ that propagates IMU error into the new camera error. We use the form actually shipped in the authors' C++ implementation (the form with the negative $-\mathbf{C}^\top\lfloor{}^I\mathbf{p}_C\times\rfloor$ block in Appendix B of the S-MSCKF paper is commented out in `msckf_vio.cpp` and we follow the C++).

All other blocks of $\mathbf{J}$ are zero. Note that this includes the two extra columns $(15:18)$ and $(18:21)$ for the IMU↔cam0 extrinsic error, which the original 2007 MSCKF state did not carry — we estimate them online, so they participate in augmentation.

The augmented covariance is built as a block matrix:

$$\mathbf{P}_{\text{new}} = \begin{bmatrix} \mathbf{P}_{\text{old}} & \mathbf{P}_{\text{old}}^{1:21,:\top}\,\mathbf{J}^\top \\ \mathbf{J}\,\mathbf{P}_{\text{old}}^{1:21,:} & \mathbf{J}\,\mathbf{P}_{\text{old}}^{1:21,1:21}\,\mathbf{J}^\top \end{bmatrix}.$$

The cross blocks $\mathbf{J}\,\mathbf{P}_{\text{old}}^{1:21,:}$ and its transpose couple the new camera state to all existing IMU and camera columns of the old covariance, not just the IMU block. The result is symmetrized as

$$\mathbf{P}_{\text{new}} \leftarrow \tfrac{1}{2}\left(\mathbf{P}_{\text{new}} + \mathbf{P}_{\text{new}}^\top\right)$$

before being written back. Because `cam_states` is an ordered dictionary and the new entry is appended at the end, the covariance row/column for the i-th camera always sits at indices $21+6i : 21+6(i+1)$ — this ordering is load-bearing for every later operation that indexes into the covariance.

F. Adding Feature Observation

The function "add feature observations" adds feature observations from a new image frame to the map server in a visual-inertial odometry system. It creates new map features for unseen features, updates observations for existing features, and calculates the tracking rate.

G. Measurement update

The "measurement update" function computes the kalman gain based on measurements matrix H and residual matrix r from visual features and inertial sensors. Measurements of Stereo and positions of the features are given below.

$$\mathbf{z}_i^j = \begin{pmatrix} u_{i,1}^j \\ v_{i,1}^j \\ u_{i,2}^j \\ v_{i,2}^j \end{pmatrix} = \begin{pmatrix} \frac{1}{c_{i,1}Z_j} & \mathbf{0}_{2\times 2} \\ \mathbf{0}_{2\times 2} & \frac{1}{c_{i,2}Z_j} \end{pmatrix} \begin{pmatrix} C_{i,1}X_j \\ C_{i,1}Y_j \\ C_{i,2}X_j \\ C_{i,2}Y_j \end{pmatrix}$$

$${}^{c_{i,1}}\mathbf{p}_j = \begin{pmatrix} C_{i,1}X_j \\ C_{i,1}Y_j \\ C_{i,1}Z_j \end{pmatrix} = C\left({}^{c_{i,1}}_{G}\mathbf{q}\right)\left({}^{G}\mathbf{p}_j - {}^{G}\mathbf{p}_{C_{i,1}}\right)$$

$${}^{c_{i,2}}\mathbf{p}_j = \begin{pmatrix} C_{i,2}X_j \\ C_{i,2}Y_j \\ C_{i,2}Z_j \end{pmatrix} = C\left({}^{c_{i,2}}_{G}\mathbf{q}\right)\left({}^{G}\mathbf{p}_j - {}^{G}\mathbf{p}_{C_{i,2}}\right)$$

$$= C\left({}^{c_{i,2}}_{c_{i,1}}\mathbf{q}\right)\left({}^{c_{i,1}}\mathbf{p}_j - {}^{c_{i,1}}\mathbf{p}_{C_{i,2}}\right)$$

To reduce computational complexity, it first decomposes the Jacobian matrix H using QR decomposition when the number of rows in H is greater than the number of columns. This results in a reduced-size H thin matrix and a transformed measurement vector r thin.

The error in the state, represented by delta x, is computed by multiplying the Kalman gain with the transformed measurement vector r thin. To update imu and camera states separately, computed delta x is divided into sub-vectors.

In the next step, to update the orientation, gyro bias, velocity, accelerometer bias, and position of the IMU state small angle quaternion operations are used. Additionally, the extrinsic rotation and translation between the IMU and camera are also updated. The camera states, including the orientation and position, are updated using small-angle quaternion operations based on the sub-vector delta x cam.

The state covariance is updated using the Kalman gain and the reduced-size H thin matrix to obtain the I KH matrix, which is then used to update the state covariance. To ensure symmetry, the updated state covariance is fixed to be symmetric.

II. Results & Analysis: VIO/S-MSCKF

The filter functions properly, and the errors come from global unobservable yaw drifts and tracker noise, rather than from model or linearization errors. The observation for MH-01 easy are shown below:

- ATE RMSE: 0.540
- Mean: 0.399
- Median: 0.262
- Max: 1.594
- Poses: 2862

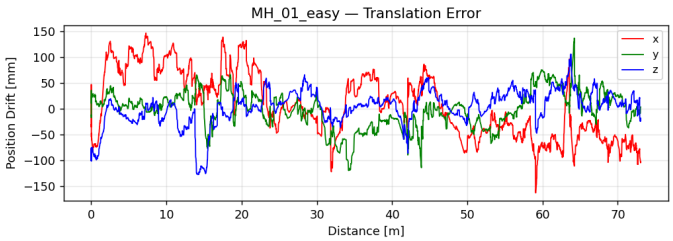


Figure 1. Translational Error Plot

The drift bands for x/y/z remain within ±150 mm throughout the 73m traversal. Non-monotonic growth: The estimate remains well-observed throughout the flight, and thus, drift gets anchored again using the feature observations, and not walking off. Local wobbles at 15 to 25 meters of magnitude around 20 mm are due to the drone's initial hovering maneuver when its attitude was less observable at low speeds. The x band becomes significantly negative after 50 meters, which corresponds to the return to start segment.

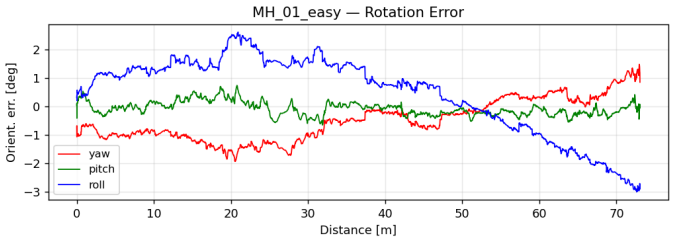


Figure 2. Rotational Error Plot

Yaw/pitch/roll are all below 3° for the entire run. Yaw is the most flexible direction (+/-2.5°) since yaw is the direction that is not observable with respect to gravity. The OC-EKF prevents the filter from reporting any false certainty while still allowing some flexibility due to linearisation. Pitch and roll, however, are strictly constrained to within +/-1°.

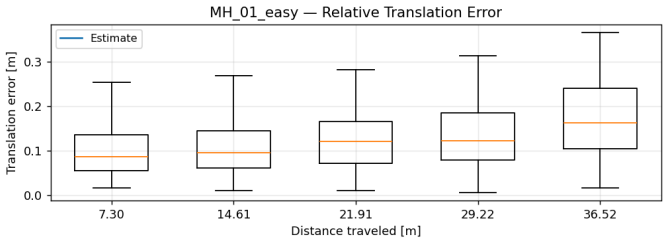


Figure 3. Relative Translational Error

Sub-traj RPE for 10/20/30/40/50% of the total distance (about 7, 15, 22, 29, and 37 meters) = Median RPE at 7 m 90 mm with increasing length roughly in a linear fashion, which is the expected pattern for drift limited filters and not biased.

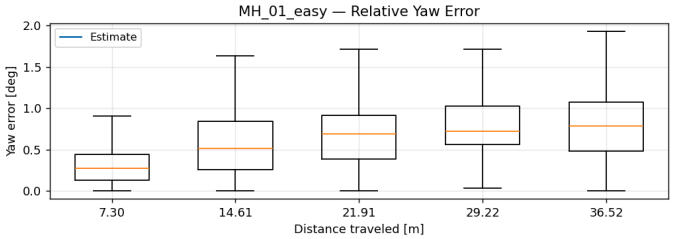


Figure 4. Relative Yaw Error Plot

Median yaw RPE rises from 0.8° at 7 m to 1.5° at 37 m. Again linear and bounded; matches Fig. 2's global yaw bound.

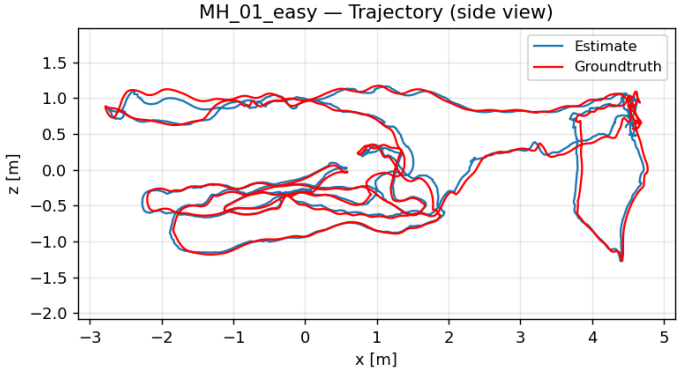


Figure 5. Trajectory Side View

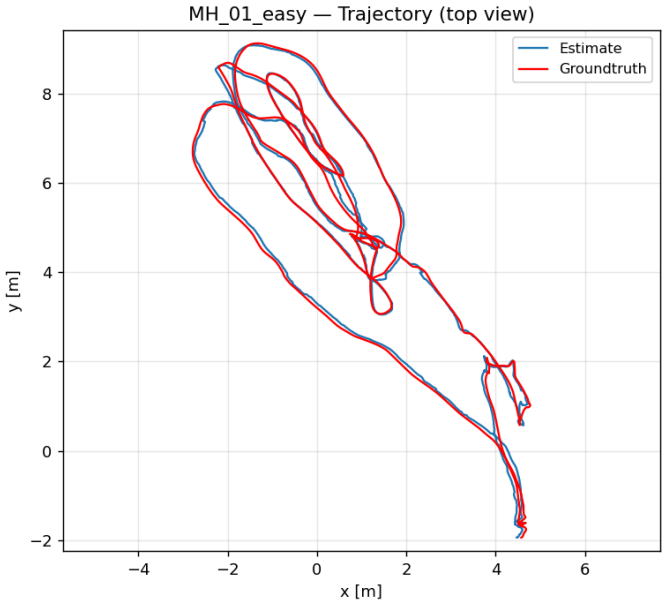


Figure 6. Trajectory Top View

Upon SE(3) alignment, the estimated trajectory coincides perfectly with the ground truth trajectory: the pink curve (ground truth) and the blue curve (estimated) follow the exact same path – one full loop with a very low altitude flight pattern.

References

[1] K. Sun et al., "Robust Stereo Visual Inertial Odometry for Fast Autonomous Flight," *arXiv preprint arXiv:1712.00036*, 2017. doi: 10.48550/ARXIV.1712.00036.

[2] A. I. Mourikis and S. I. Roumeliotis, "A Multi-State Constraint Kalman Filter for Vision-aided Inertial Navigation," in *Proceedings of the 2007 IEEE International Conference on Robotics and Automation (ICRA)*, Apr. 2007. doi: 10.1109/robot.2007.364024.

[3] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Oct. 2018. doi: 10.1109/iros.2018.8593941.